2.4. Building networks with PyTorch

There are many deep learning frameworks available today, with TensorFlow, MXNet, and PyTorch probably being the most popular. We chose to use PyTorch for this book because of its simplicity. It allows you to write native-looking Python code and still get all the goodies of a good framework like automatic differentiation and built-in optimization. We'll give you a quick introduction to PyTorch here, but we'll explain more as we go along. If you need to brush up on basic deep learning, see the appendix where we have a fairly detailed review of deep learning and more thorough coverage of PyTorch.

If you're comfortable with the numpy multidimensional array, you can replace almost everything you do with numpy with PyTorch. For example, here we instantiate a 2×3 matrix in numpy:

```
>>> import numpy

>>> numpy.array([[1, 2, 3], [4, 5, 6]])
array([[1, 2, 3],
       [4, 5, 6]])
```

And here is how you instantiate the same matrix with PyTorch:

```
>>> import torch

>>> torch.Tensor([[1, 2, 3], [4, 5, 6]])
1  2  3
4  5  6
[torch.FloatTensor of size 2x3]
```

The PyTorch code is basically the same as the numpy version, except in PyTorch we call multidimensional arrays *tensors*. Unsurprisingly, this is also the term used in TensorFlow and other frameworks, so get used to seeing multidimensional arrays referred to as tensors. We can and do refer to the *tensor order*, which is basically how many indexed dimensions the tensor has. This gets a little confusing because sometimes we speak of the dimension of a vector, in which case we're referring to the length of the vector. But when we speak of the order of a tensor, we mean how many indices it has. A vector has one index, meaning every element can be